SOFTWARE ENGINEERING ACADEMIC PROJECT — MID-SEMESTER EVALUATION

Department of Computer Science & Engineering

System Architecture Design

Project: AutoTestHub (Web Automation & Testing Platform)

MULTI-TIER ARCHITECTURE • IPC PROTOCOLS • SUBSYSTEM TOPOLOGY

ARCHITECTURAL METADATA

System: AutoTestHub

Architecture Pattern: Decoupled Multi-Tier Subprocess Architecture

IPC Protocol: JSON over Stdin / Stdout Streams

Storage: MongoDB 7.0+ & Local Artifact Store

PROJECT ENGINEERING TEAM

Shristi Neekhra (Integration & Execution Service)

Shifa Siddiqui (Backend Architecture & Database)

Shruti Gupta (Selenium Engine & Worker Module)

Shriya Agarwal (Frontend Control Desk & Dashboard)

Supervisor: [Supervisor Name]

1. Architectural Overview & Topology

AutoTestHub employs a decoupled, four-tier architecture designed to isolate the high-overhead, synchronous requirements of browser automation from the asynchronous event loop of web APIs. This decoupling ensures that heavy browser sessions, unpredictable web latencies, and unexpected DOM crashes never block backend responsiveness or compromise system stability.

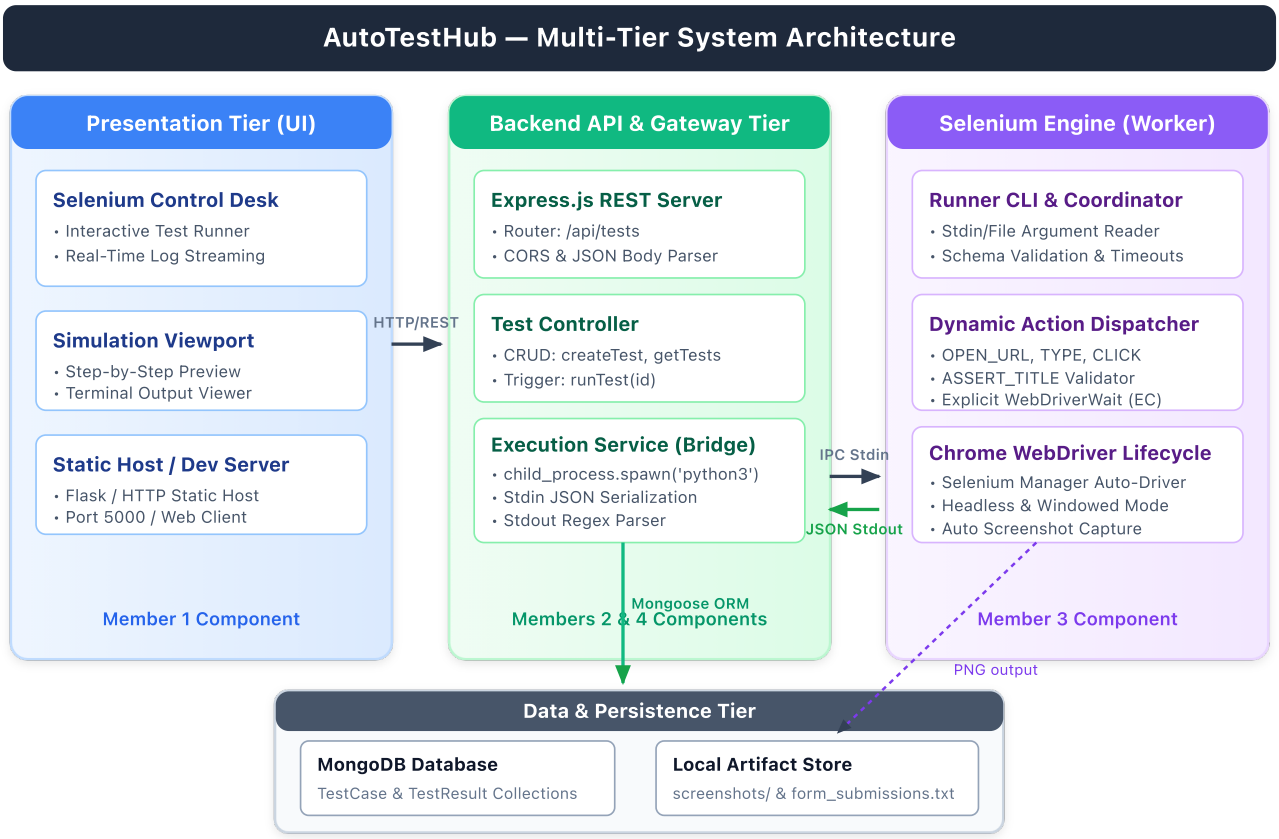


Figure 1: High-level Multi-Tier System Architecture and Inter-Process Communication Flow.

1.1 Architectural Tiers & Subsystem Responsibilities

Tier 1: Presentation & Client Dashboard

The presentation layer consists of the **Selenium Control Desk**, built with modern semantic HTML5, CSS3, and vanilla JavaScript. It offers two distinct operational modes:

- **Live Execution Mode:** Dispatches execution commands to the backend, opening real Chrome browser windows and receiving streaming progress updates.
- **Client Simulation Mode:** Steps through automation actions within an in-page browser viewport mock, allowing developers to dry-run scripts without browser overhead.

Tier 2: Backend API & Gateway Layer

The API gateway is implemented using **Node.js and Express.js**. It provides CORS-enabled REST endpoints, request body parsing, input validation, and routing. The gateway manages test CRUD operations and delegates execution triggers to the execution service.

Tier 3: Asynchronous Execution Bridge

The `executionService.js` module serves as the mission-critical bridge between the Node.js event loop and the Python automation worker. When a test is triggered, the service:

1. Spawns an isolated `python3 runner.py` child process via Node's `child_process.spawn()`.
2. Serializes the Mongoose test case document into a sanitized JSON string and streams it into the child process's `stdin`.
3. Buffers `stdout` and `stderr` independently.
4. Applies a robust regular expression match (`/\{[\s\S]*\}/`) on `stdout` upon process closure to extract the final result JSON, immune to extraneous driver log noise.
5. Persists the parsed outcome in MongoDB and handles fallback error logging if unexpected crashes occur.

Tier 4: Selenium Automation Engine (Worker Module)

The core test executor is implemented in **Python 3.12** utilizing **Selenium 4**. Key architectural highlights include:

- **Selenium Manager:** Eliminates third-party binary managers by natively discovering installed Google Chrome binaries and downloading the corresponding ChromeDriver automatically.
- **Dynamic Action Dispatcher:** Maps declarative action types (`OPEN_URL`, `TYPE`, `CLICK`, `ASSERT_TITLE`) to modular action handlers.
- **Explicit Synchronization:** Uses `WebDriverWait` with `expected_conditions` to guard against asynchronous UI rendering delays.
- **Fail-Safe Lifecycle:** Guarantees `driver.quit()` execution within `finally` blocks and auto-captures PNG screenshots for both successful and failing runs.

1.2 Inter-Process Communication Contract

The contract between the Node.js Execution Service and Python Worker is strictly defined:

Input Payload (Piped via Stdin):

```
{
  "testCaseId": "6ad5f308777225dd7c9950a4",
  "name": "Integration Test Pipeline",
  "url": "https://example.com",
  "actions": [
    { "type": "OPEN_URL" },
    { "type": "ASSERT_TITLE", "expected": "Example Domain" }
  ]
}
```

Output Response (Emitted as Single-Line JSON to Stdout):

```
{
  "testCaseId": "6ad5f308777225dd7c9950a4",
  "status": "PASSED",
  "executionTime": 1.48,
  "screenshot": "screenshots/6ad5f308777225dd7c9950a4_20261019_160809_9fb515.png",
  "message": "Page title matched 'Example Domain'",
  "errorMessage": null
}
```